

数据管线实验指南

第005单元 从13行记录建立可审计的输入张量

你将建立数据字典、识别重复导入、按ID连接标签、固定三份实体名单、绘制训练分布与逐条样本图，并验证只有训练数据可以决定填补值与缩放刻度。完成的是一个数据入口实验；不用GPU、网络、API或神经网络框架。

先修为004。实验最小运行环境为Python 3.12与NumPy 2.3.5。Notebook与画图另需JupyterLab、ipykernel、nbformat、Matplotlib和可显示中文的Noto Sans CJK字体。可以在已有课程环境运行，也可以执行下列命令建立独立环境。environment.yml声明版本；制作时没有重新测试每一种操作系统上的Anaconda安装。

```
conda env create -f environment.yml
conda activate dl-unit-005
python experiment.py
python test_experiment.py
jupyter lab experiment.ipynb
```

打开Notebook后重启内核，再从第一格顺序运行到底，不要只运行最后一格。命令须在units/005目录执行；脚本本身通过文件所在目录定位数据，--output-dir可指定结果目录。图像生成程序需要系统已安装上述中文字体；若缺少会明确报错，数值管线本身不依赖字体。

一 先在纸上确定信息边界

主任务是在每天09:00根据两个旋钮预测校准最终读数，关心未用于训练的新设备。post_reading和标签均在09:05可用，因此不准进入X。主数据只有前两天，采用跨设备留出，不能据此证明第三天或更久以后的表现。

读取data/README.md，在不运行程序前回答：一行表示什么？唯一观测键是什么？哪一列允许空白？哪两列按什么顺序进入X？把时间和用途写成四列表格。如果你的答案没有预测时点，先补齐再继续。

文件records_raw.csv共有13行，最后一行是A02的完全相同副本。labels.csv有12条标签，顺序与输入相反。entity_split.json保存正式名单；time_illustration.json只解释未来时间留出，明确第三天数据不存在，不能拿它当正式三份拆分文件。

二 读入 去重 连接

运行前先预测下面代码会输出什么，然后在Notebook核对。

```

from pathlib import Path
import experiment as e
raw = e.read_csv(e.R00T / "data/records_raw.csv")
rows, removed = e.clean_records(raw)
labels = e.read_csv(e.R00T / "data/labels.csv")
joined = e.join_labels(rows, labels)
print(len(raw), len(rows), removed)
print(joined[0]["record_id"], joined[0]["target"])

```

预期是13、12、["A02"]，以及A01对应2。保留A01和A02两次合法观测，不能按设备去重。查看原始标签的第一条，会看到F02对应47；如果直接按位置配对，A01会被错标47，但数组长度仍然匹配。

在内存副本中把重复的A02旋钮1改成99，再调用clean_records。应出现包含“conflicting record_id”的ValueError。不要改原始CSV来“修复”错误。失败测试的目的，是确认冲突被挡在边界，而不是为了得到一条能跑完的错误数据。

三 固定名单并写出shape

正式训练ID为A01、A02、B01、B02、C01、C02；验证ID为D01、D02；测试ID为E01、E02、F01、F02。读取名单并调用split_records。写出三个输入矩阵的shape，应为(6,2)、(2,2)、(4,2)。

```

import json
lists = json.loads((e.R00T / "data/entity_split.json").read_text())
parts = e.split_records(joined, lists)
X_train = e.feature_matrix(parts["train"])
print(X_train)
print(X_train.shape)

```

检查每条观测恰好出现一次，并检查设备集合分别为{A,B,C}、{D}、{E,F}。把A02与D01在名单中的位置交换，行名单仍不重复，但设备会跨集合。主任务模式应拒绝这样的拆分。不要把require_new_entities关闭来掩盖错误；只有任务改成允许同实体时，才应重新设计相应时间条件。

四 手算描述与预处理

训练旋钮1为1、2、3、4、5、6；旋钮2为10、缺失、30、40、50、60。手算已观测个数、总和、平均、最小最大值。旋钮2缺失比例应为1/6；平均的分母则为5。把这两个分母写清楚后才运行describe。

```

print(e.describe(X_train))
state = e.fit_preprocessor(X_train)
filled_train, scaled_train = e.transform(X_train, state)
print(state)
print(scaled_train)

```

预期mean为[3.5,38], low为[1,10], high为[6,60], span为[5,50]。A02填补后是[2,38], 缩放后是[0.2,0.56]。D01缩放后是[1.2,1.2], D02是[1.4,0.56]。验证超过1不自动说明错误, 它表示超出训练极值。

使用训练保存的state转换验证和测试, 不要给每份数据分别fit。再把验证旋钮1改成70000, 检查旧state没有变化; 重新走正式训练拟合时也应该没有变化。这个“不受留出值影响”的测试, 比仅检查mean看起来合理更有力。

五 画出能发现问题的图

执行make_figures.py或Notebook中的作图格。先观察训练旋钮1的直方图: 区间[1,3)、[3,5)、[5,7]应各有2条。边界3进入第二格, 5进入第三格。更改区间时必须同时记录新边界, 不要把柱子高度变化当作数据变化。

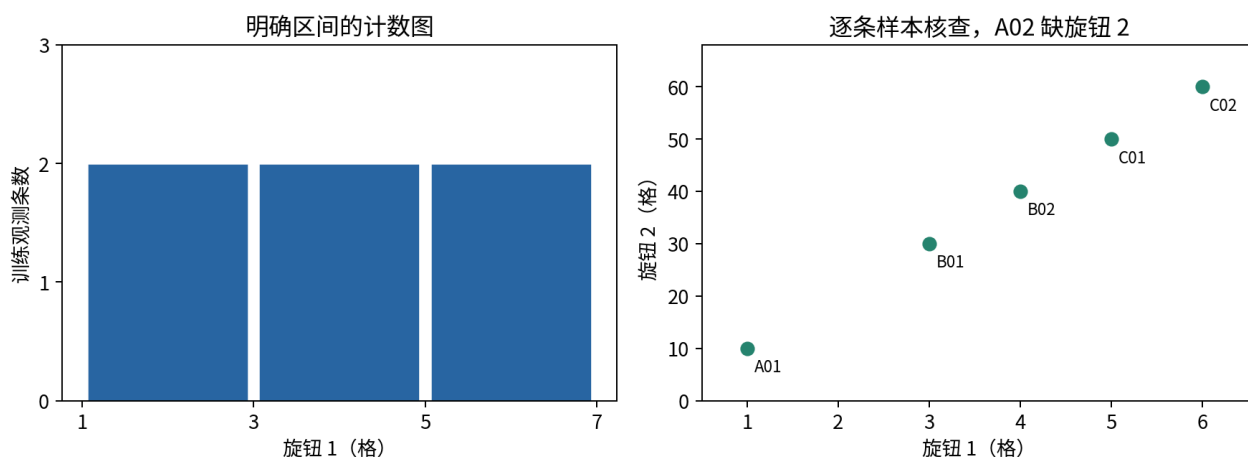


图1 正式训练输入的计数与逐条观测。右图只有五个点, 因为A02缺第二个旋钮; 缺失不应被画在0的位置冒充观测。

逐条样本图需要单位、ID和缺失说明。用图回查CSV里的C02是否确为(6,60), 并说明A02为什么没有坐标点。若填补后再画, 应注明“填补值”, 不能把它与原始测量混称。这里的样本是表格观测, 因此“样本图”是逐条散点, 不需要外部照片。

六 独立运行三个故意错误

第一, 传入特征列("knob1", "post_reading")。feature_matrix应拒绝, 因为未来字段不在允许名单。第二, 单独演示用12条唯一观测拟合: 旋钮2平均为68, 而非38。这个错误示范只能写入带wrong标记的审计字段, 不得覆盖正式state。

第三, 运行memorization_demo。行级同实体留出MAE为0, 新实体留出MAE为71/3。重新手算训练A/B/C的平均6和三个新设备误差11、23、37。回答: 两个拆分测量的是不是同一个部署问题? 不能把它们当作一场公平算法竞赛。

主脚本已明确分开正式结果与错误演示，并设置`test_targets_scored`为`false`。这不是一个真正未见的盲测，整个教材数据都公开可读；这项标记只说明主流程没有计算主数据测试目标的误差，不表示作者或读者从没看过数据。

七 边界测试与提交物

运行`test_experiment.py`应通过9组测试，其中包括40组独立Fraction有理数核对。测试还会拒绝标签重复或缺项、空数据、错误shape、无穷输入、全缺失训练列、常量训练列、列顺序改变、未来字段和实体重叠。测试失败时先定位阶段，不要删掉失败断言。

提交以下内容：你的数据字典与预测时点说明；固定三份名单；训练描述与手算；带单位和ID的两种图；一条失败输入及错误解释；可顺序运行的Notebook；正式运行产生的`preprocessor`、`tensor_ledger`、`audit`、`split_lists`和`environment`文件。

`experiment.py`默认写入`outputs`，Notebook写入`notebook_outputs`。两者的正式数组、名单和参数应相同。环境文件记录Python、NumPy版本和数据文件SHA-256；摘要不同先查输入版本。不要把旧输出复制成新运行的证据。

验收不以“运行没报错”为终点。必须能指出A02的原始缺失、训练填补值38、缩放值0.56、它所属的训练名单，以及为什么D02也沿用38。最后用自己的话解释：若任务改为已知设备未来预测，当前实体名单为什么不足以回答新问题。